

Modül 04: Metotlar

Metot bildirmek sürpriz içermez. Bu modüldeki üç şey içerir: Java'nın argümanları nasıl geçirdiği, aşırı yüklemeler arasında nasıl seçim yaptığı ve `static` kelimesinin gerçekte ne anlama geldiği. Bunlardan ilki dildeki en çok yanlış anlaşılan konudur.

Neler Öğreneceksiniz

- Java'nın nesneler için de neden her zaman değer geçişi yaptığı ve bundan ne çıktığı
- Aşırı yükleme çözümleme sırası ve `List` içinde yol açtığı gerçek hata
- Dönüş tipinin neden metot imzasının parçası olmadığı
- Varargs ve `null` değerinin dönüşüm gerektirdiği tek durum
- `static` ne zaman doğru, ne zaman sizi sonradan engelliyor

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Aynı ad, farklı parametreler	mümkün değil, varsayılan kullanın	aşırı yükleme
Varsayılan parametre değerleri	<code>def f(x=1)</code>	yok, onun yerine aşırı yükleyin
İsimli argümanlar	<code>f(x=1)</code>	yok
Değişken sayıda argüman	<code>*args</code>	<code>int... nums</code>
Argüman geçişi	nesneye referans, ad yeniden bağlama yerel	aynı semantik, farklı sözcükler

Son satır önemli. Bir nesne geçirdiğinizde Python ve Java **aynı** davranır. İkisi de fonksiyonun nesneyi değiştirmesine izin verir, ikisi de çağırının değişkenini yeniden bağlamasına izin vermez. Tartışma yalnızca iki topluluğun "referans geçişi" ifadesini farklı anlamlarda kullanmasından çıkıyor.

Ders

Değer geçişi, kesin olarak

Metot **değişkenin bir kopyasını** alır. Değişken bir referans tuttuğunda metot referansın kopyasını alır, dolayısıyla ikisi de aynı nesneyi gösterir.

`PassByValue.java` aynı görünen ama olmayan iki durumu gösteriyor. Gerçek çıktı:

```
primitive after the call: 10
list after mutation:      [original, added inside the method]
list after reassignment:  [original]
string after the call:    before
```

Aynı tip, aynı çağrı biçimi, zıt sonuçlar:

```
void mutateTheObject(List<String> items) {
    items.add("added inside the method"); // changes the shared object
}

void reassignTheParameter(List<String> items) {
    items = new ArrayList<>();           // rebinds the local copy only
}
```

Birincide metod, referansının kopyasını takip edip tek paylaşılan nesneye ulaştı ve onu değiştirdi. İkincide kendi kopyasını başka bir yere yöneltti. Sizin değişkeniniz hiç kıpırdamadı, çünkü metodun onu kıpırdatmanın bir yolu yok.

Gerçekten kullanacağınız üç sonuç:

- Bir metodun verinizi değiştirebilmesini istiyorsanız değiştirilebilir nesne geçirin.
- İstemiyorsanız değişmez bir şey, `List.copyOf` ya da savunmacı kopya geçirin.
- Bir metod değişkeninizi asla yeniden yöneltmez. Size farklı bir nesne vermesi gerekiyorsa onu **döndürmek** zorundadır.

String'ler bu tartışmayı karıştırır çünkü değişmezdirler, dolayısıyla onlar için değiştirme durumu hiç yoktur. Her String metodu yeni bir nesne döndürür.

Aşırı yükleme çözümlemesi

Hiçbir aşırı yükleme tam eşleşmediğinde derleyici şu sabit sırayı dener:

```
1. genişletme  int -> long -> float -> double
2. kutulama    int -> Integer
3. varargs     int -> int...
```

İlk işe yarayanı alır ve **derleme** zamanında karar verir. `Overloading.java` dosyasından:

```
pick(1)    -> long        genişletme kutulamayı yendi
choose(1)  -> Integer     kutulama varargs'ı yendi
```

Bu sıra geriye dönük uyumluluk için var. Genişletme ve varargs otomatik kutulamadan eskidir, dolayısıyla kutulamayı tercih etmek Java 5 öncesi yazılmış kodun anlamını değiştirdi.

Bunun yol açtığı hata

`List` hem `remove(int index)` hem `remove(Object o)` metoduna sahip:

```
after remove(1):           [10, 30]
after remove(Integer.valueOf(30)): [10]
```

Birincisi **indeks 1'deki** elemanı sildi, ki o `20` idi. `1` değerini değil.

Bu gerçek üretim hatalarına yol açmıştır. Bir `List` içinde `Integer` tutulduğunda hangi aşırı yüklemeyi kastettiğinizi her zaman açıkça belirtin.

Dönüş tipi imza değildir

```
int    parse(String s) { ... }
double parse(String s) { ... }
```

```
error: method parse(String) is already defined in class ReturnTypeError
```

Derleyici aşırı yüklemeyi çağrı yerindeki argümanlardan seçer. Tek başına bir ifade olarak `parse("1");` yazıldığında seçim yapacağı bir şey yoktur. Standart kütüphanenin `parse` adında iki metot yerine `parseInt` ve `parseDouble` içermesinin sebebi tam olarak budur.

Varargs

Metodun içinde varargs parametresi yalnızca bir dizidir. `Varargs.java` :

```
int count(int... numbers) { return numbers.length; }
```

Üç kural:

1. Son parametre olmalı ve yalnızca bir tane olabilir.
2. Argümansız çağırmak **boş dizi** verir, asla `null`. Sıradan çağrılar için null kontrolü gerekmez.
3. Açıkça verilen `null` belirsizdir ve dönüşüm ister:

```
describe((Object[]) null) -> the array itself was null
describe((Object) null)   -> an array of length 1
```

`String.format` ve `printf` varargs metotlarıdır, herhangi sayıda değer almalarının sebebi bu.

static

`static` sınıfa aittir. Diğer her şey bir nesneye aittir.

Baştan beri fark etmeden örnek metotları yazıyorsunuz: kompakt kaynak dosyasının `main` metodu bir tanesi, içinde `this` çalışmasının sebebi bu.

`static` kullanın: metodun belirli bir nesneden hiçbir şeye ihtiyacı yoksa, `Math.max` gibi argümanlarının saf bir fonksiyonuysa, ya da `List.of` gibi bir fabrikaysa.

Kaçının: metot nesne başına duruma dokunuyorsa, ya da sonradan geçersiz kılmak isteyebilirsiniz.

Statik metotlar çok biçimli değildir ve Modül 06 bunun bedelini anlatıyor.

Faydalı bir test: bir metodu statik yapmak, bir nesnenin zaten bileceği birkaç şeyi parametre olarak geçirmeyi gerektiriyorsa, o metot statik olmamalıdır.

Statik *durum* tüm program için tek paylaşılan kopyadır. Kullanışlı, ve çok iş parçacıklı bir programda patlamayı bekleyen bir yarış koşulu. Modül 18 buna geri dönüyor.

Çalıştırın

```
java modules/04-methods/examples/PassByValue.java
java modules/04-methods/examples/Overloading.java
java modules/04-methods/examples/Varargs.java
java modules/04-methods/examples/StaticVsInstance.java

# Bilerek başarısız olur.
java modules/04-methods/examples/ReturnTypeIsNotASignature.java

./scripts/verify-examples.sh modules/04-methods
```

Sık Yapılan Hatalar

Bir metodun değişkeninizi yeniden yönelteceğini beklemek. Yapamaz. Yeni değeri döndürmesini sağlayın.

`List<Integer>` üzerinde `list.remove(someInt)` çağırmak. Değeri değil konumu sildiniz. Sarın: `list.remove(Integer.valueOf(x))`.

Getter'dan iç koleksiyonunuzu vermek. Çağırان artık nesnenizin durumuna bir referans tutuyor ve arkanızdan değiştirebilir. Önemli olduğunda `List.copyOf(items)` döndürün.

Yalnızca dönüş tipiyle aşırı yüklemeye çalışmak. Metotlara farklı ad verin.

Varsayılan parametre değeri beklemek. Java'da yok. Daha dolu sürümü çağıran bir aşırı yükleme yazın:

```
String greet(String name) { return greet(name, "Hello"); }
String greet(String name, String word) { return word + ", " + name; }
```

"Şu an bir nesneye ihtiyacı yok" diye static yapmak. Bir metodu statik yapmak kolay, çağırانlar ona bağlandıktan sonra geri almak zahmetlidir, özellikle sonradan geçersiz kılmak ya da taklit etmek istediğinizde.

Özet Çıkarımlar

- **Java her zaman değer geçişi yapar.** Kopyalanan değişkendir, ve nesnelerde değişken bir referanstır.
- **Parametrenin nesnesini değiştirmek çağırana görünür. Parametreyi yeniden atamak görünmez.** Bu tek ayırım tüm tartışmayı çözer.
- **Aşırı yüklemeler genişletme, sonra kutulama, sonra varargs sırasıyla çözümlenir,** derleme zamanında.
- `List.remove(int)` indeks alır, `remove(Object)` değer alır. Liste `Integer` tuttuğunda açık olun.
- **Dönüş tipi imzanın parçası değildir.** Farklı dönüş değil, farklı ad.
- **Varargs bir dizidir,** en sonda olmalıdır ve null değil boşdur.
- `static` nesne yok demektir, dolayısıyla `this` yok ve geçersiz kılma yok.

Ödev

[homework/README.md](#)

Kendisine verileni değiştiremeyen bir metot yazın, kanıtlayın, sonra `List.remove` tuzağına bilerek düşün. Referans çözüm [solutions/04-methods/](#) altında.